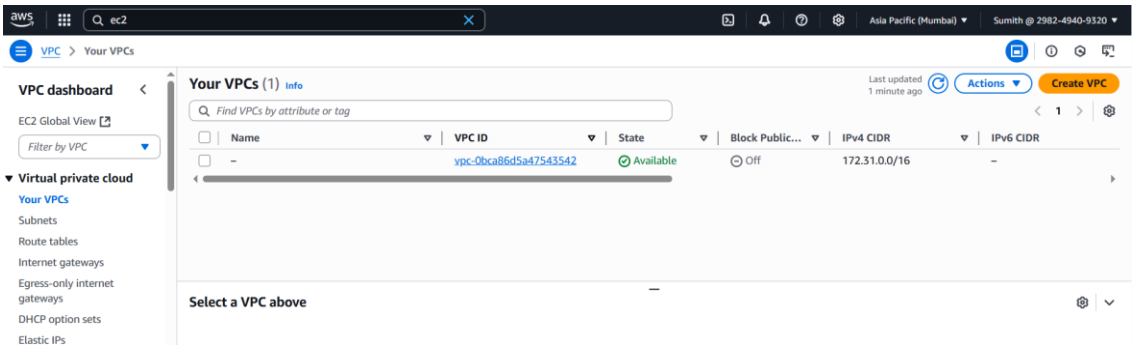
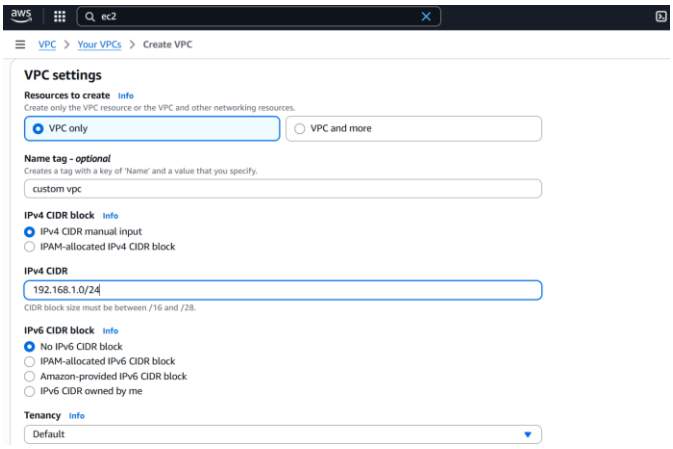
# TASKS :

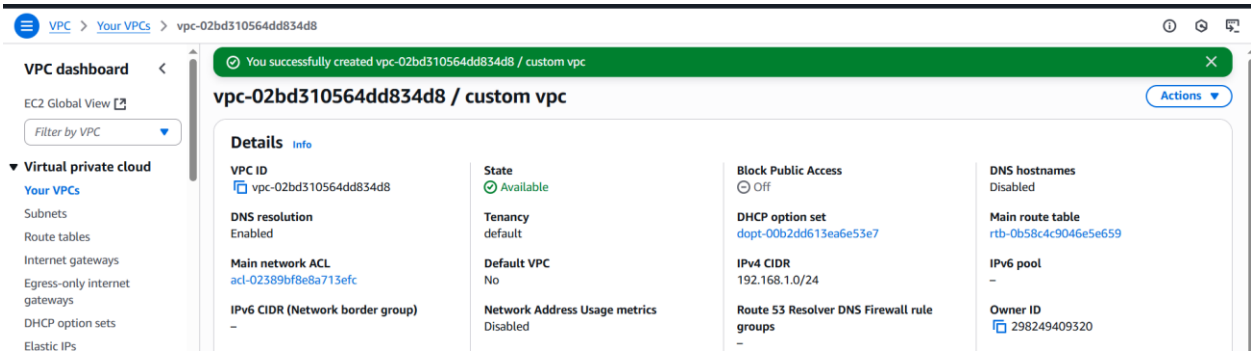
## ✅ 1. VPC Setup

### A. Create a Custom VPC

- CIDR Block: 10.0.0.0/16
- Name: Demo-VPC

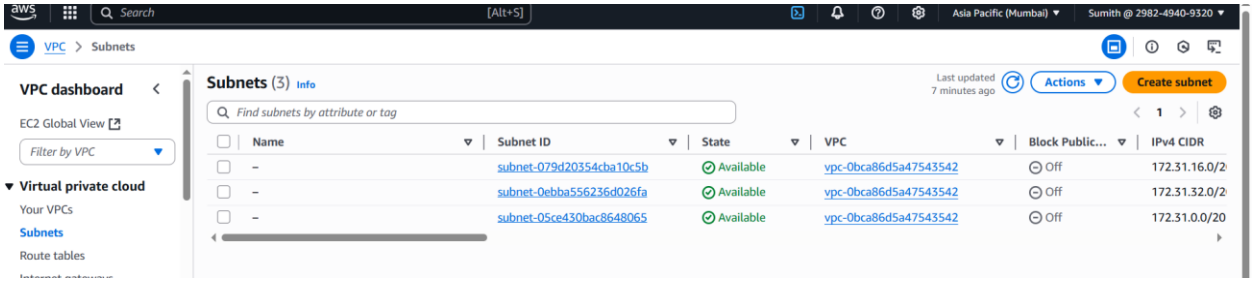


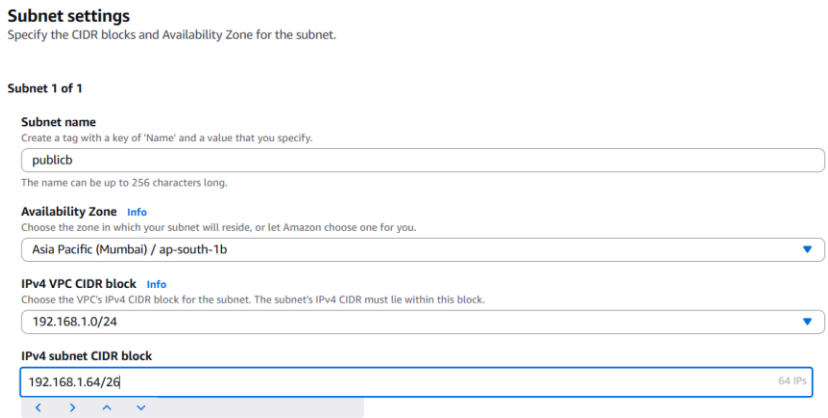



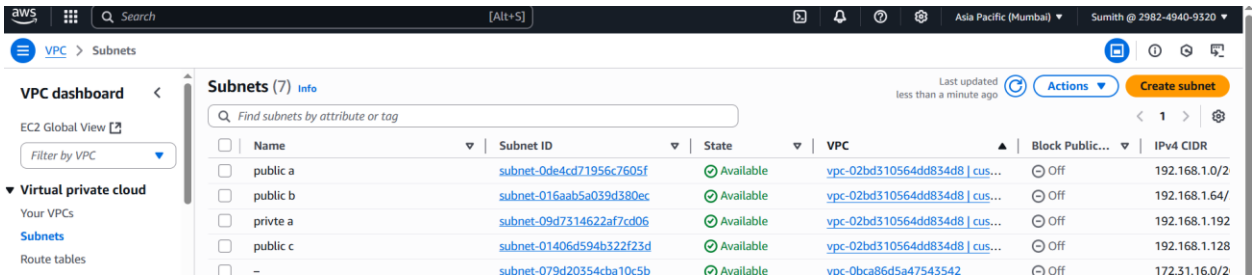



## B. Create Subnets

1. **Public Subnet**
   - CIDR: 10.0.1.0/24
   - Availability Zone: ap-south-1a (or your preferred zone)
   - Auto-assign Public IP: **Enable**
   - Name: Public-Subnet
2. **Private Subnet**
   - CIDR: 10.0.2.0/24
   - Availability Zone: ap-south-1a
   - Name: Private-Subnet



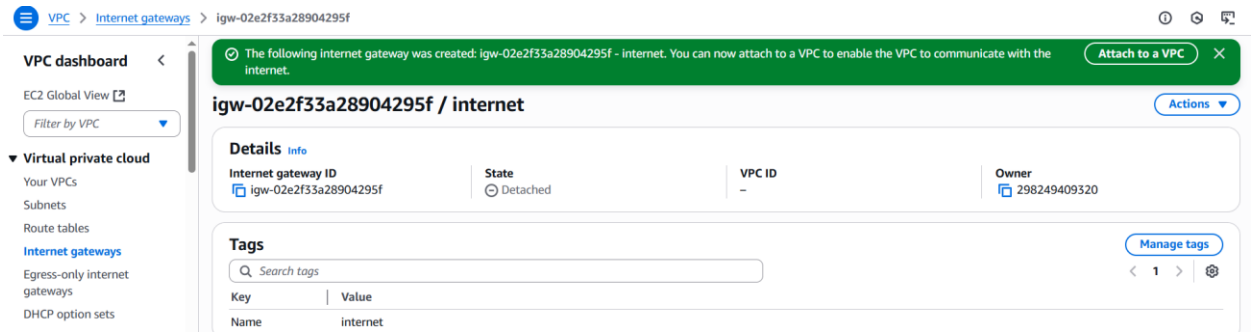



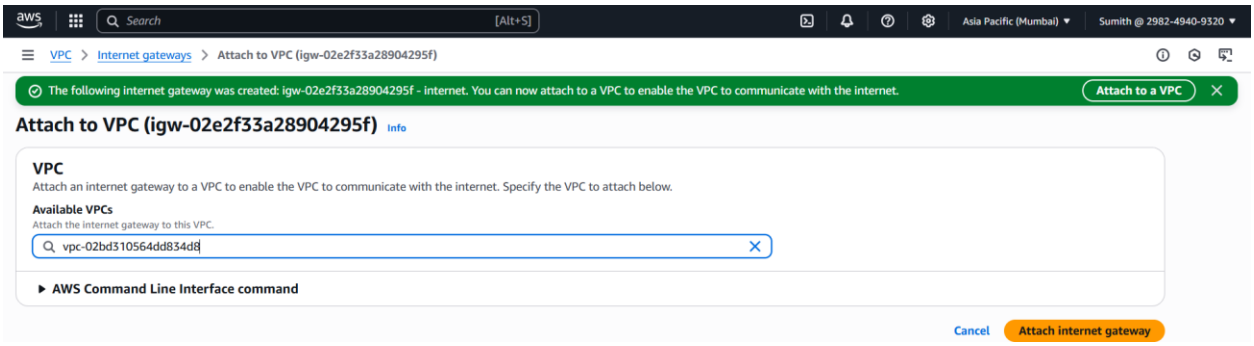



## ✅ 2. Configure Internet Access

### A. Create and Attach Internet Gateway

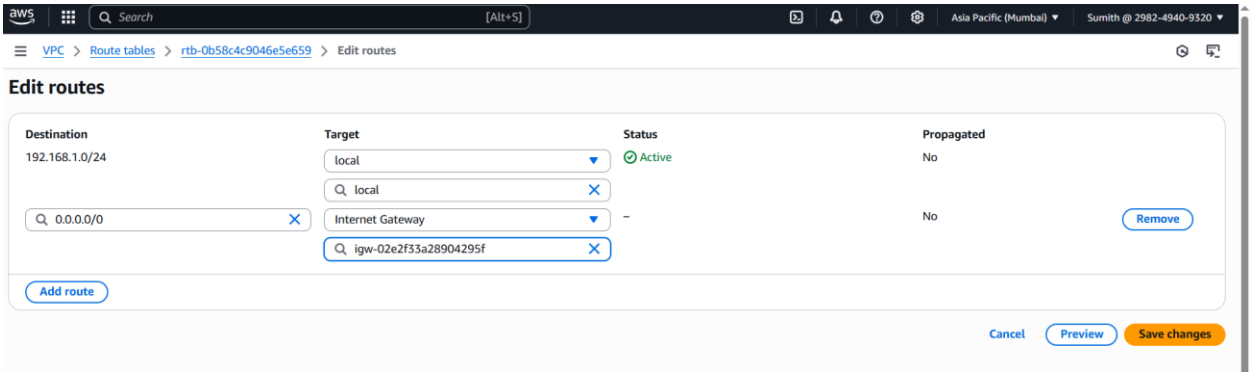
- Name: Demo-IGW
- Attach to: Demo-VPC

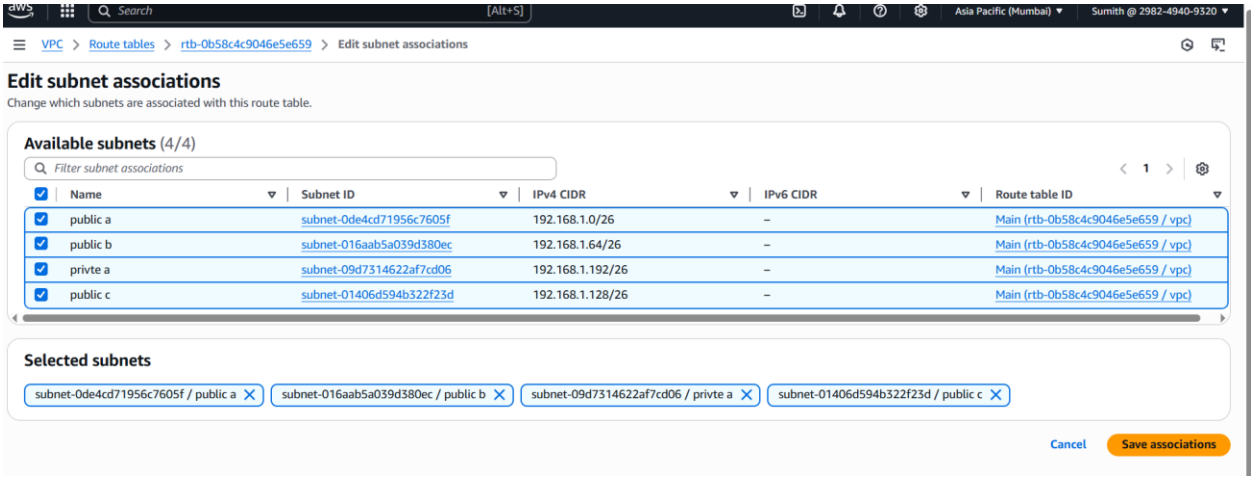


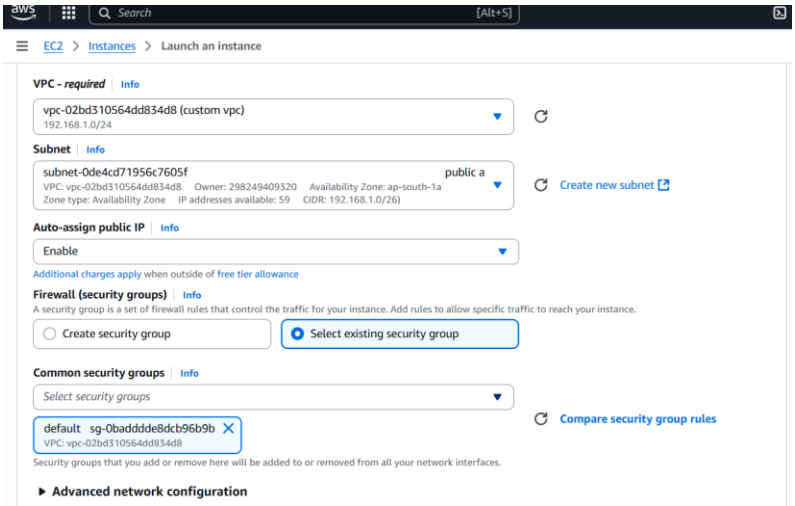




### B. Public Route Table

- Name: Public-RT
- Add Route:
  - Destination: 0.0.0.0/0
  - Target: Internet Gateway (Demo-IGW)
- Associate with: Public-Subnet






---

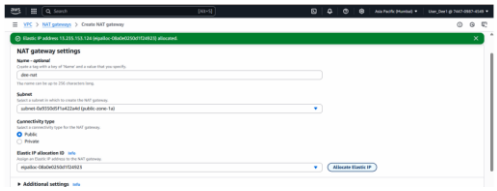
## ✅ 3. Create NAT Gateway (For Private Subnet)

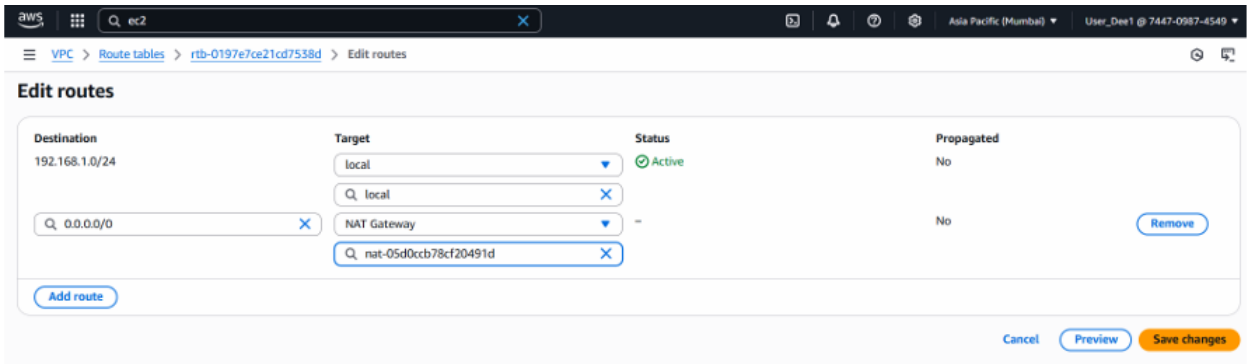
### A. Allocate Elastic IP

- Name: NAT-EIP

### B. Create NAT Gateway

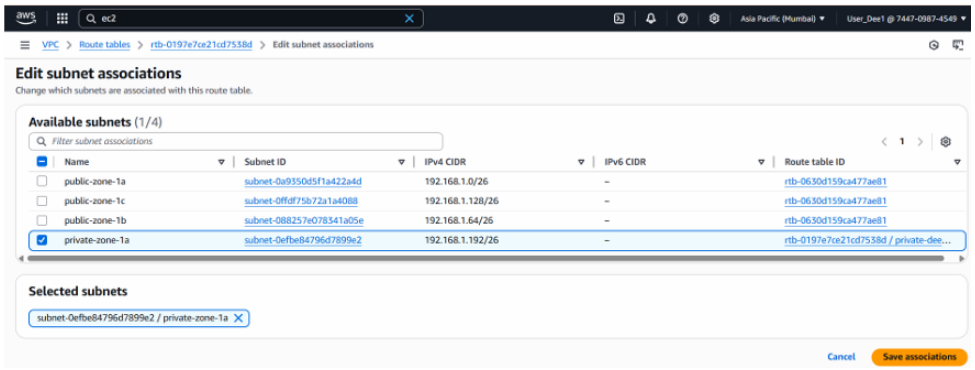
- Name: Demo-NATGW
- Subnet: Public-Subnet
- Elastic IP: NAT-EIP






### C. Private Route Table

- Name: Private-RT
- Add Route:
  - Destination: 0.0.0.0/0
  - Target: NAT Gateway (Demo-NATGW)
- Associate with: Private-Subnet

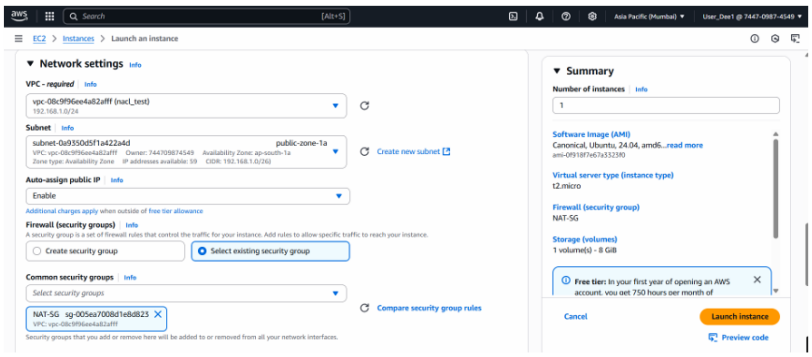



---

## ✅ 4. Launch EC2 Instances

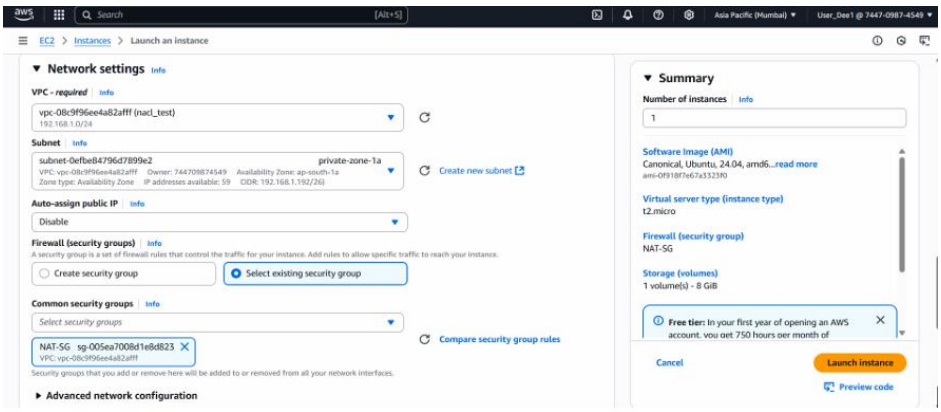
### A. Public EC2 Instance

- AMI: Amazon Linux 2
- Subnet: Public-Subnet
- Auto-assign Public IP: **Yes**
- Security Group: Allow **SSH (22)** and **ICMP (ping)**
- Name: Public-EC2




**B. Private EC2 Instance**

- AMI: Amazon Linux 2
- Subnet: Private-Subnet
- Auto-assign Public IP: **No**
- Security Group: Allow **SSH from Public-EC2's SG** and **ICMP**
- Name: Private-EC2




---

## ✅ 5. Verification

**A. SSH into Public EC2**

bash

CopyEdit

```
ping 8.8.8.8    # Should work
```

```
ubuntu@ip-192-168-1-16:~$ ping 8.8.8.8
PING 8.8.8.8 (8.8.8.8) 56(84) bytes of data.
64 bytes from 8.8.8.8: icmp_seq=1 ttl=117 time=1.96 ms
64 bytes from 8.8.8.8: icmp_seq=2 ttl=117 time=2.21 ms
64 bytes from 8.8.8.8: icmp_seq=3 ttl=117 time=2.23 ms
64 bytes from 8.8.8.8: icmp_seq=4 ttl=117 time=2.24 ms
64 bytes from 8.8.8.8: icmp_seq=5 ttl=117 time=2.23 ms
64 bytes from 8.8.8.8: icmp_seq=6 ttl=117 time=2.06 ms
^C
--- 8.8.8.8 ping statistics ---
6 packets transmitted, 6 received, 0% packet loss, time 5006ms
rtt min/avg/max/mdev = 1.963/2.154/2.235/0.104 ms
ubuntu@ip-192-168-1-16:~$
```

**B. SSH into Private EC2 via Public EC2 (Bastion)**

bash

CopyEdit

# From Public-EC2:

ssh -i your-key.pem ec2-user@<Private-EC2-IP>

ping 8.8.8.8  # Should work only after NAT Gateway is set up

```
ubuntu@ip-192-168-1-16:~$ ssh -i "NAT-keyPair.pem" ubuntu@192.168.1.202
The authenticity of host '192.168.1.202 (192.168.1.202)' can't be established.
ED25519 key fingerprint is SHA256:qOzvenaEaW49Ph375/ifPqqQ9D7Xcvc+NYTGsI/qyDo.
This key is not known by any other names.
Are you sure you want to continue connecting (yes/no/[fingerprint])? yes
Warning: Permanently added '192.168.1.202' (ED25519) to the list of known hosts.
@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@
@         WARNING: UNPROTECTED PRIVATE KEY FILE!          @
@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@
Permissions 0664 for 'NAT-keyPair.pem' are too open.
It is required that your private key files are NOT accessible by others.
This private key will be ignored.
Load key "NAT-keyPair.pem": bad permissions
ubuntu@192.168.1.202: Permission denied (publickey).
ubuntu@ip-192-168-1-16:~$ chmod 400 NAT-keyPair.pem
```

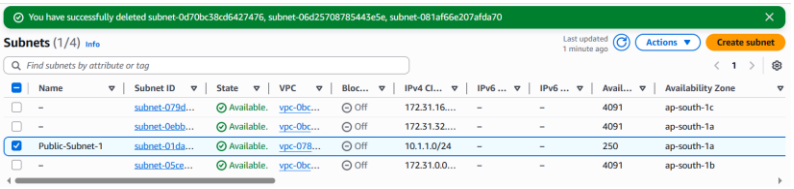
```
ubuntu@ip-192-168-1-202:~$ ping 8.8.8.8
PING 8.8.8.8 (8.8.8.8) 56(84) bytes of data.
```

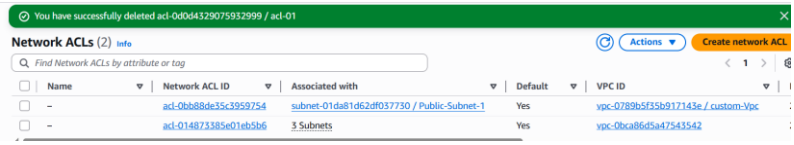
After NAT Gateway Setup:

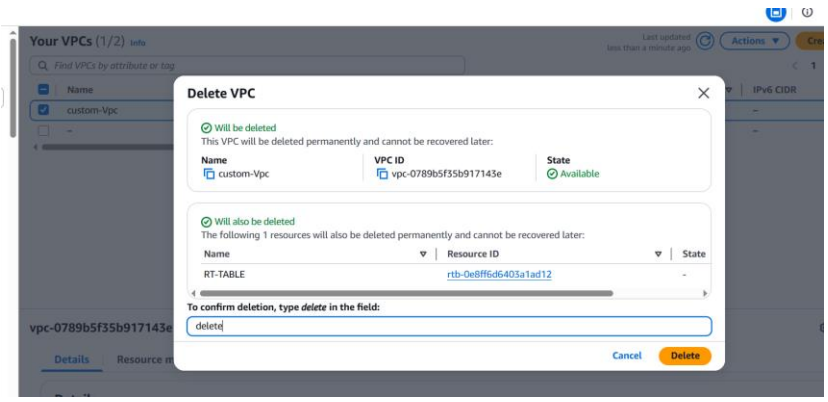
```
ubuntu@ip-192-168-1-202:~$ ping 8.8.8.8
PING 8.8.8.8 (8.8.8.8) 56(84) bytes of data.
64 bytes from 8.8.8.8: icmp_seq=1 ttl=113 time=2.10 ms
64 bytes from 8.8.8.8: icmp_seq=2 ttl=113 time=1.78 ms
64 bytes from 8.8.8.8: icmp_seq=3 ttl=113 time=1.83 ms
64 bytes from 8.8.8.8: icmp_seq=4 ttl=113 time=1.78 ms
64 bytes from 8.8.8.8: icmp_seq=5 ttl=113 time=2.41 ms
^C
--- 8.8.8.8 ping statistics ---
5 packets transmitted, 5 received, 0% packet loss, time 4007ms
rtt min/avg/max/mdev = 1.775/1.979/2.414/0.247 ms
ubuntu@ip-192-168-1-202:~$
```

## ✅ 6. Cleanup (Avoid Charges)

- Terminate both EC2 Instances
- Delete:
  - NAT Gateway (first disassociate EIP)
  - Elastic IP
  - Subnets
  - Route Tables
  - Internet Gateway
  - VPC



## NAT Gateway vs. Internet Gateway – Comparison:

| Feature | Internet Gateway (IGW) | NAT Gateway (NGW) |
|---|---|---|
| **Purpose** | Enables internet access for **public** resources (with public IP) | Enables internet access for **private** resources (without public IP) |
| **Direction of traffic** | **Inbound & Outbound** | **Outbound only** |
| **Works with** | EC2s in **public subnets** (must have public IP) | EC2s in **private subnets** (no public IP) |
| **IP Address required** | Yes (Elastic IP for public EC2s) | Yes (Elastic IP for NAT Gateway itself) |
| **Managed by** | AWS-managed, simple to attach | AWS-managed, but incurs additional cost |
| **Security** | Must use security groups and NACLs | Must route only trusted outbound traffic |
| **Typical use case** | Hosting web servers, load balancers, public APIs | Allowing private EC2s to install patches, download packages |

🔷 **Why must a NAT Gateway be placed in a Public Subnet?**

A **NAT Gateway** needs to:

1. **Access the Internet** to route outbound traffic for private instances.
2. Therefore, it **must**:
   - Be placed in a **public subnet**.
   - Be attached to an **Elastic IP**.
   - Be reachable **via an Internet Gateway**.

✅ **Logic:**

A **private instance** cannot go directly to the internet. It sends its outbound traffic to the **NAT Gateway** (in a public subnet), which **then accesses the Internet on its behalf** and relays the response back.

 **Analogy:**

Think of the NAT Gateway as a **proxy in a public office** that makes calls to the outside world **on behalf of** employees in a secured basement (private subnet).

---

◆ **Bastion Host – Enhancing Security**

✅ **What is a Bastion Host?**

A **Bastion Host** is a secure, publicly accessible EC2 instance in a **public subnet**, used to **SSH (or RDP)** into **private instances** in a **private subnet**.

🔐 **How it Enhances Security:**

| Feature | Description |
|---|---|
| **Controlled Entry Point** | Only this instance has SSH access from the internet. |
| **No public IPs in private subnet** | Keeps private EC2s isolated from direct internet access. |
| **Tightly controlled security** | Access allowed only from specific IP ranges (e.g., your office/home IP). |
| **Can be monitored/audited** | Logs of access can be collected from the bastion for auditing purposes. |

🛡️ **Without Bastion:** Each private EC2 would need a public IP + SSH access — **increasing attack surface**.

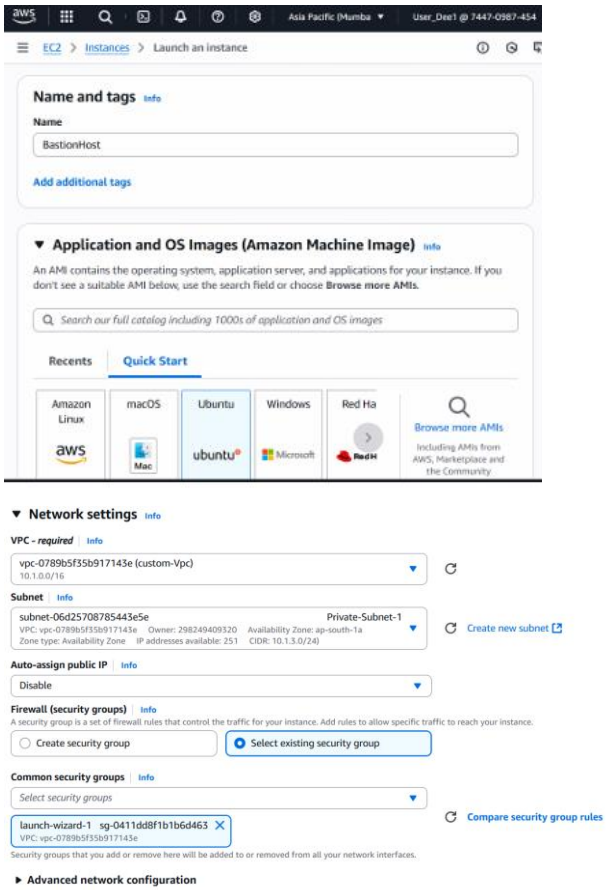
🛡️ **With Bastion:** Only one entry point exists. Security groups and firewalls are **much easier to manage**.

## Task 1: Set up a Bastion Host to SSH into the Private EC2 Instance:

🔧 **Step-by-Step Setup**

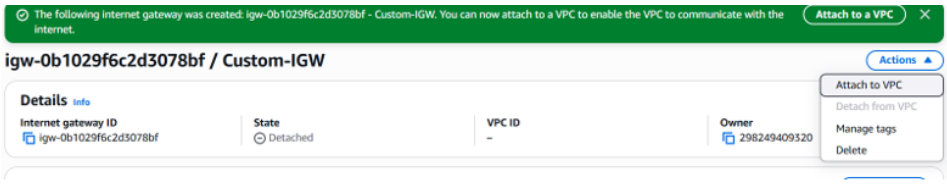
**Step 1: Create a VPC (if not created already)**

- **Create a custom VPC (e.g., 10.0.0.0/16)**
- **Add two subnets:**
  - **Public Subnet: 10.0.1.0/24**
  - **Private Subnet: 10.0.2.0/24**




**Step 2: Create & Attach Internet Gateway**

- **Go to VPC > Internet Gateways**
- **Create one, and attach it to your VPC**
- **Edit the public subnet's route table:**
  - **Destination: 0.0.0.0/0 → Target: Internet Gateway**



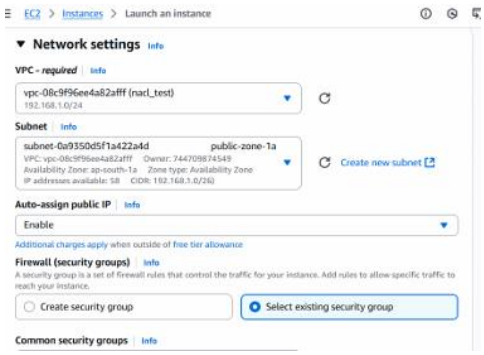

---

**Step 3: Launch EC2 – Bastion Host (Public EC2)**

- **Subnet: Public Subnet**
- **Assign Public IP: Enable**
- **Security Group:**

- **Inbound rule: SSH (port 22) → From: Your IP only (e.g., 203.0.113.5/32)**

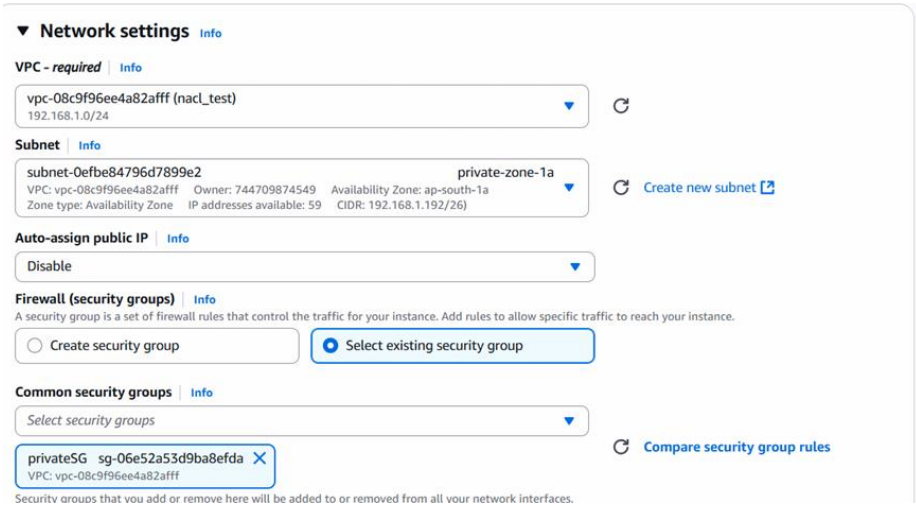
- **Key Pair: Use or create .pem key (e.g., bastion-key.pem)**




---

**Step 4: Launch EC2 – Private Instance**

- **Subnet: Private Subnet**
- **Assign NO Public IP**
- **Security Group:**
  - **Inbound rule: SSH (port 22) → From: Bastion Host's Security Group**




---

**Step 5: SSH Flow (Your Laptop → Bastion → Private EC2)**

**✅ From your terminal:**

**bash**

**CopyEdit**

**# Step 1: Connect to the Bastion**

**ssh -i bastion-key.pem ec2-user@<bastion-public-ip>**

**# Step 2: From inside Bastion → Connect to private EC2**

**ssh -i bastion-key.pem ec2-user@<private-instance-private-ip>**

```
ubuntu@ip-192-168-1-44:~$ ssh -i "NAT-keyPair.pem" ubuntu@192.168.1.230
@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@
@         WARNING: UNPROTECTED PRIVATE KEY FILE!          @
@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@
Permissions 0664 for 'NAT-keyPair.pem' are too open.
It is required that your private key files are NOT accessible by others.
This private key will be ignored.
Load key "NAT-keyPair.pem": bad permissions
ubuntu@192.168.1.230: Permission denied (publickey).
ubuntu@ip-192-168-1-44:~$ chmod 400 NAT-keyPair.pem
ubuntu@ip-192-168-1-44:~$ ssh -i "NAT-keyPair.pem" ubuntu@192.168.1.230
Welcome to Ubuntu 24.04.2 LTS (GNU/Linux 6.8.0-1029-aws x86_64)

 * Documentation:  https://help.ubuntu.com
 * Management:     https://landscape.canonical.com
 * Support:        https://ubuntu.com/pro

 System information as of Mon Jul 28 09:29:40 UTC 2025

  System load:  0.08              Processes:             103
  Usage of /:   25.3% of 6.71GB   Users logged in:       0
  Memory usage: 21%               IPv4 address for enX0: 192.168.1.230
  Swap usage:   0%

Expanded Security Maintenance for Applications is not enabled.

0 updates can be applied immediately.
```

```
ubuntu@ip-192-168-1-97: ~

Ubuntu comes with ABSOLUTELY NO WARRANTY, to the extent permitted by
applicable law.

To run a command as administrator (user "root"), use "sudo <command>".
See "man sudo_root" for details.

ubuntu@ip-192-168-1-10:~$ ping 8.8.8.8
PING 8.8.8.8 (8.8.8.8) 56(84) bytes of data.
64 bytes from 8.8.8.8: icmp_seq=1 ttl=117 time=1.73 ms
64 bytes from 8.8.8.8: icmp_seq=2 ttl=117 time=1.87 ms

^C--- 8.8.8.8 ping statistics ---
2 packets transmitted, 2 received, 0% packet loss, time 1002ms
rtt min/avg/max/mdev = 1.733/1.802/1.872/0.069 ms
ubuntu@ip-192-168-1-10:~$ sftp -i "bkp.pem" ubuntu@43.205.195.92
Warning: Identity file bkp.pem not accessible: No such file or directory.
The authenticity of host '43.205.195.92 (43.205.195.92)' can't be established.
ED25519 key fingerprint is SHA256:bE51CxUrT5AShSO00IgxCo21BXIUUNLU/1gIszhiCyA.
This key is not known by any other names.
Are you sure you want to continue connecting (yes/no/[fingerprint])? yes
Warning: Permanently added '43.205.195.92' (ED25519) to the list of known hosts.
ubuntu@43.205.195.92: Permission denied (publickey).
Connection closed.
Connection closed
ubuntu@ip-192-168-1-10:~$ exit
logout
Connection to 43.205.195.92 closed.

C:\Users\Sujith\Downloads>sftp -i "bkp.pem" ubuntu@43.205.195.92
Connected to 43.205.195.92.
```

🛡️ **Best Practices**

- **Don't open port 22 to 0.0.0.0/0 — always restrict to trusted IPs.**
- **Disable root login on the bastion.**
- **Enable CloudWatch logging or use Session Manager for audit trails.**
- **Use different keys for bastion and private EC2s in production.**
- **Rotate keys regularly.**

**Task 2: Test Cross-VPC Communication Using VPC Peering (Optional)**

---

🔷 **Objective**

**Allow instances in VPC-A to communicate with instances in VPC-B via VPC Peering.**

---

🔹 **Step-by-Step Process**

🔹 **1. Create Two VPCs**

| VPC Name | CIDR Block |
|---|---|
| VPC-A | 10.0.0.0/16 |
| VPC-B | 10.1.0.0/16 |

**Each VPC should have at least one public subnet and one EC2 instance launched.**

---

🔹 **2. Create VPC Peering Connection**

- **Go to VPC Dashboard → Peering Connections → Create Peering**
  - **Requester: VPC-A**
  - **Accepter: VPC-B (same or another AWS account)**
- **After creation, click Actions → Accept Request**

---

🔹 **3. Update Route Tables**

**To allow traffic between VPCs:**

- **Go to VPC-A's route table, add:**
  - **Destination: 10.1.0.0/16 → Target: Peering Connection**
- **Go to VPC-B's route table, add:**
  - **Destination: 10.0.0.0/16 → Target: Peering Connection**

---

### ◈ 4. Modify Security Groups

**For both EC2 instances, allow traffic from the other VPC:**

- **Inbound Rule:**
  - **Type: ICMP (ping) or SSH**
  - **Source: VPC CIDR (e.g., 10.1.0.0/16)**

---

### ◈ 5. Test the Connection

**From EC2 in VPC-A:**

**bash**

**CopyEdit**

```
ping <Private IP of EC2 in VPC-B>
```

**Or SSH:**

**bash**

**CopyEdit**

```
ssh -i key.pem ec2-user@<private-ip-of-other-vpc-instance>
```

---

### ⭘ Limitations of VPC Peering:

- **No transitive peering**
- **Cannot use overlapping CIDRs**
- **One-to-one communication only**

---

## ✅ Task 3: Explore Network ACLs vs Security Groups

---

### ◆ Comparison Table

| Feature | Security Group | Network ACL (NACL) |
|---|---|---|
| Level | Instance-level | Subnet-level |
| Stateful? | ✅ Yes (automatically allows return traffic) | ❌ No (must manually allow return traffic) |
| Default behavior | Deny all unless allowed | Allow all unless denied (default ACL) |
| Rules evaluated | All rules are evaluated | Rules are evaluated in number order |
| Rule types | Allow only | Allow and Deny |
| Use case | Control access to individual EC2s | Control access to entire subnets |
| Recommended for | Everyday access control | Fine-grained subnet-level filtering (e.g., deny IP ranges) |

---

### ◈ Example Use Cases

| Scenario | Use Security Group? | Use NACL? | Why? |
|---|---|---|---|
| Allow SSH to EC2 from office IP | ✅ Yes | ❌ No | Per-instance control |
| Block specific IP ranges from entire subnet | ❌ No | ✅ Yes | Subnet-level block |
| Allow HTTP from everywhere | ✅ Yes | ❌ No | Easier in SG |
| Deny all outbound traffic | ❌ No | ✅ Yes | NACLs support DENY |

---

### ▯ Example NACL Rule Setup (Allow HTTP only)

| Rule # | Type | Protocol | Port | Source | Allow/Deny |
|---|---|---|---|---|---|
| 100 | HTTP | TCP | 80 | 0.0.0.0/0 | ALLOW |
| 110 | All | ALL | ALL | 0.0.0.0/0 | DENY |

➡ **Must add both inbound and outbound rules separately!**